



OPEN Modeling and optimization of bakery production scheduling to minimize makespan and oven idle time

Majharulislam Babor^{1✉}, Olivier Paquet-Durand¹, Reinhard Kohlus² & Bernd Hitzmann¹

Makespan dominates the manufacturing expenses in bakery production. The high energy consumption of ovens also has a substantial impact, which bakers may overlook. Bakers leave ovens running until the final product is baked, allowing them to consume energy even when not in use. It results in energy waste, increased manufacturing costs, and CO₂ emissions. This paper investigates three manufacturing lines from small and medium-sized bakeries to find optimum makespan and ovens' idle time (OIDT). A hybrid no-wait flow shop scheduling model considering the constraints that are most common in bakeries is proposed. To find optimal solutions, non-dominated sorting genetic algorithm (NSGA-II), strength Pareto evolutionary algorithm (SPEA2), generalized differential evolution (GDE3), improved multi-objective particle swarm optimization (OMOPSO), and speed-constrained multi-objective particle swarm optimization (SMPSO) were used. The experimental results show that the shortest makespan does not always imply the lowest OIDT. Even the optimized solutions have up to 231 min of excess OIDT, while the makespan is the shortest. Pareto solutions provide promising trade-offs between makespan and OIDT, with the best-case scenario reducing OIDT by 1348 min while increasing makespan only by 61 min from the minimum possible makespan. NSGA-II outperforms all other algorithms in obtaining a high number of good-quality solutions and a small number of poor-quality solutions, followed by SPEA2 and GDE3. In contrast, OMOPSO and SMPSO deliver the worst solutions, which become pronounced as the problem complexity grows.

Bakery is one of the major food manufacturing sectors, with steady increases in market share and per capita consumption. Craft bakery sales in Germany in 2021 were 14.9 billion Euros (exclusive of VAT), with an increase of 0.18 billion Euros per year. To meet market demand, the amount of flour consumed, the variety of products developed, and the number of personnel employed have all expanded over the past decade. According to reports, each bakery uses on average of 372 MWh of energy annually, resulting in 101 tons of CO₂ emissions^{1,2}. As the business environment has become more competitive, the objectives for improving the efficiency of a manufacturing system have widened. In order to satisfy customers, meet market demand, and turn a profit, an optimum cost-time profile is crucial. It includes cost savings via the efficient use of assets and materials. Makespan, tardiness, earliness, and energy consumption are some of the most commonly employed cost-cutting objectives in various production environments. However, bakery manufacturing, particularly in small and medium-sized bakeries, is prone to inefficiencies because employees perform many tasks manually for operations that cannot be automated. Furthermore, employee salaries are said to account for a significant amount of the cost¹. As a result, when planning the production schedule, bakers focus primarily on lowering the makespan.

Production scheduling with more than two machines is a non-deterministic polynomial-time (NP)-hard problem^{3,4}. The difficulty of finding the best schedule increases as the number of products, processing stages, and alternative machines for each stage grows. Therefore, the flow shop scheduling problem has been extensively studied to improve the efficiency of several production and service environments, such as bakery^{5–7}, glass⁸, steel⁹, wood¹⁰, chemical process¹¹, energy system¹², healthcare system^{13–15}. To put it simply, it is the process of allocating tasks of varying durations from n products to m machines. It also provides supplementary information for assessing a schedule, such as makespan and energy use, that change based on how the tasks are allocated. The most

¹Institute of Food Science and Biotechnology, Department of Process Analytics and Cereal Science, University of Hohenheim, 70599 Stuttgart, Germany. ²Institute of Food Science and Biotechnology, Department of Process Engineering and Food Powders, University of Hohenheim, 70599 Stuttgart, Germany. ✉email: majhar@uni-hohenheim.de

common type of flow shop scheduling problem is the permutation flow shop, in which each product must pass through all the m machines independently in the same order⁴. In complex cases, a processing task of a product may depend on another product. It is known as “no-wait” flow shop scheduling problem when there is no delay allowed between two successive tasks of a product. Many hybrid flow shop models have been developed to reflect the reality, which is mostly specific to a production system¹⁶. To make production systems energy-efficient and environmentally friendly, many flow shop models have been proposed, which are widely known as “green flow shop model”. Here, in order to establish an efficient resource allocation, total energy consumption is taken into account in addition to makespan¹⁷.

Although modern industries have been applying many powerful decision-making tools, such as intelligent manufacturing systems, to address complex challenges^{18,19}, small and medium-sized bakeries continue to rely on personal experience^{6,7}. Furthermore, bakeries’ product range and amount change frequently due to market demand and seasonality, demanding continuous monitoring of production efficiency. However, only a few studies have focused on improving bakery manufacturing. In a recent study, Huber and Stuckenschmidt²⁰ implemented machine learning approaches to predict hourly sales of bakery items in a retailer store and optimize the baking schedule so that bakers serve customers with fresh products. Nonetheless, the production of a vast number of products, from flour to finished or unfinished goods before delivery to retailers, is a separate segment. In a case study with a medium-sized German bakery, Hecker et al.⁶ observed that the makespan of an existing manufacturing line with 40 products can be lowered by 8.6%. Swangnop et al.⁷ developed a scheduling model for a bakery in Thailand and demonstrated that the existing production, planned based on experience, is inefficient. An additional factor that most bakers overlook is the ovens’ energy consumption, which has a vital influence on manufacturing costs and CO₂ emissions. It has been reported that only baking consumes up to 78% of the total energy depending on the product category²¹. Bakeries typically feature multiple ovens with varied functionalities that are employed according to product specifications^{5,6}. As a result, by minimizing the idle time of the ovens, a large quantity of energy can be saved, lowering manufacturing costs and CO₂ emissions. Babor et al.⁵ investigated a small Spanish bakery and observed that actual production is poorly optimized. The authors weighed the machines’ idle time and makespan to the objective function.

The motivation for this study is twofold. First, to solve production optimization problems for bakeries with two objectives: minimizing makespan and energy waste due to oven idle time. It is a hybrid no-wait flow shop scheduling problem because of the following exceptions in bakery manufacturing. Many tasks are carried out manually, and there are numerous substitute machines that can carry out the remaining tasks. Additionally, there are production constraints for a variety of products recipes. Therefore, a mixed-integer linear programming approach for hybrid no-wait flow shop scheduling model (HNFSM) is proposed to simulate bakery production scheduling. The Pareto optimal solutions obtained by multi-objective optimization algorithms are used to analyze the trade-offs between the objectives. Secondly, to compare the performance of five multi-objective optimization algorithms to solve the instances. Because production optimization is time-consuming and performed frequently, attaining optimal solutions in the lowest computation time is essential. We used multi-objective optimization algorithms of two types: evolutionary algorithms and particle swarm optimization-based metaheuristics. Non-dominated sorting genetic algorithm (NSGA-II), strength Pareto evolutionary algorithm (SPEA2), and generalized differential evolution (GDE3) are taken from the former category, while improved multi-objective particle swarm optimization (OMOPSO), and speed-constrained multi-objective particle swarm optimization (SMPSO) are from the latter. To assess their effectiveness, four quality indicators are used: cardinality, convergence, distribution and spread, and convergence and distribution of the obtained solutions. To cluster the solutions into distinct qualities, a Gaussian mixture model²² is used.

The following are the contributions of the current study. State-of-the-art multi-objective optimization methods are used to optimize the production efficiency of small and medium-sized bakeries employing a hybrid no-wait flow shop model. By combining various performance metrics, the Gaussian mixture model is used to assess how effectively algorithms solve problems of three complexity levels while varying the number of products and predecessor constraints.

The remainder of the paper is structured as follows. An introduction to bakery production is given in the next section. “[Materials and methods](#)” section describes mathematical formulation of a hybrid no-wait flow shop scheduling model to simulate bakery manufacturing. Besides, multi-objective optimization algorithms and their performance indicators are presented. In “[Results and discussion](#)” section, the effectiveness of algorithms in solving scheduling problems for bakeries is analysed. “[Conclusions and future works](#)” section of this paper provides a summary of findings.

Bakery production

A bakery product undergoes a series of processing steps. Each product has a recipe that specifies the order, duration, and machines that will be utilized to complete the tasks. Figure 1 shows a simplified processing route. Making the dough starts by mixing and kneading the ingredients, such as flour, water, and salt. In most cases, yeast is added to induce fermentation, which produces the leavening agent CO₂ and other aroma precursors. If small and medium-sized bakeries are considered, the transfer of unfinished products from one machine to another is performed manually by employees almost after every processing task. The fermentation process has a vital impact on the final product’s quality. Because temperature and humidity have a considerable impact on yeast sugar fermentation, the duration of the processing stages under various conditions is strictly controlled. Performing one task longer than the predefined duration results in overtreatment and consequently, delay for the following tasks, and a loss of product quality, both of which are undesirable. Therefore, as soon as ingredients are mixed and kneaded, the next stages are carried out with no delay.

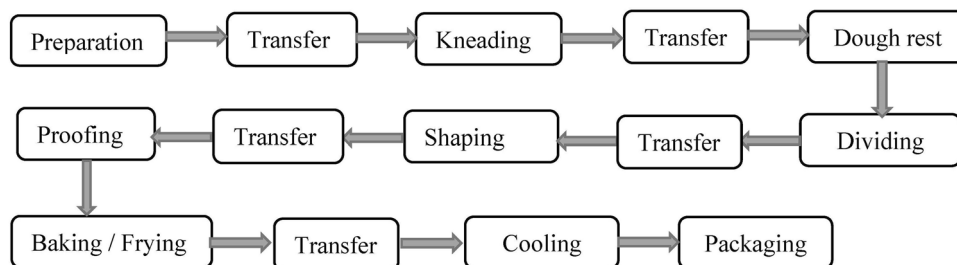


Figure 1. A simplified processing route for bakery products.

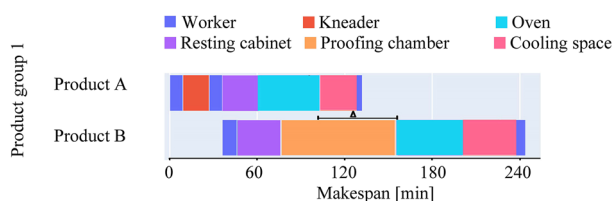


Figure 2. Gantt chart showing schedule of one product group with unified initial stages. The triangle (Δ) shows the oven idle time.

To avoid wasting energy, bakers turn off machines during the idle time—the time between two scheduled operations. However, ovens need preparation time to reach the set temperature before performing baking. Turning them off after an operation requires a restart well ahead of the next operation. When they are turned off, no energy is consumed, but the temperature steadily declines. If the idle time between two tasks is short, the temperature drops less, and the time required to re-heat is reduced. However, if the temperature drops sharply, such as due to chilly weather, the assumed time may not be long enough to reach the set point. Again, with the prolonged idle period, the right time to restart the ovens must be considered to avoid wasting energy and have them ready at the proper time. When the number of products is large and there are many manual tasks to perform, it is difficult for bakers to keep track. The following tasks must be postponed accordingly if the oven's temperature is not up to the set point in time. It may affect the product quality and lead to inefficient production. To avoid these consequences, bakers keep the ovens running throughout the production time.

The duration and machine set up for processing steps are predetermined. Hence, the energy consumption during operational time is constant regardless of how optimized a schedule is. In contrast, the effectiveness of the schedule influences oven idle time, which has a direct impact on the quantity of wasted energy. Moreover, small, medium-sized, and medium-sized manufacturers have limitations in recording energy data for each device. In this case, the idle time of the ovens can be an ideal indicator of energy waste. Machine idle time has been investigated in various studies as one of the objectives for optimizing production schedule^{5,6,23,24}.

Materials and methods

In this paper, three bakery production optimization problems, labelled with BK15, Bk40 and BK50 were solved. The number in labels specifies how many products were produced in each manufacturing line, for example, the dataset BK15 contains production information for 15 bakery products. BK15 and BK50 were taken from Babor and Hitzmann²⁵ and BK40 was taken from Hecker et al.⁶. BK15 had three employees, eight machines, and two ovens with four compartments; BK40 had eleven machines, three ovens, and nine compartments; and BK50 had ten employees, fourteen machines, and three ovens, and ten compartments. In bakeries, many ovens have separate compartments, each of which can be used independently to bake a batch of products. In the following discussion, the problems are labelled according to the approximate number of total products. The implementation and simulation of HNFSM and multi-objective optimization algorithms were performed using the computer language Python (version 3.7)²⁶ on a computer running Microsoft Windows 10 as the operating system with a configuration of an Intel Core i5 at 4 × 3.20 GHz, 8 GB ram.

Problem definition and scheduling model. In small, and medium-sized bakeries, using the same dough for various products made from the same ingredients is a frequent practice. This practice takes advantage of the machines' capabilities in the initial stages to reduce preparation time. Bakers split the dough after completing a few processing tasks into various parts. It enables the products to be treated differently in subsequent phases to meet recipe requirements. There is no common procedure for separating dough as it completely depends on the type of products and recipes. Figure 2 illustrates a schedule of two products that are produced from the same dough and shared the same processing machines at the initial phase.

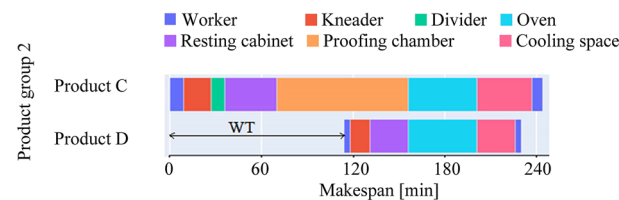


Figure 3. Simplified schedule for one product group where baking is performed together. Here WT is the difference between the start time of Product C and Product D.

Group	Product	Bowl time [min]	Name	Processing stage	Duration [min]	Machine/Employee
1	1	0	Product A	Preparation	9	Employee
				Kneading	18	Kneader
				Dividing	9	Employee
				Dough rest	25	Resting cabinet
				Baking	42	Oven
				Cooling	25	Cooling space
				Packaging	4	Employee
	2	36	Product B	Dividing	10	Employee
				Dough rest	30	Resting cabinet
				Proofing	80	Proofing chamber
				Baking	45	Oven
				Cooling	36	Cooling space
				Packaging	7	Employee

Table 1. Simplified production data for one product group.

In another scenario, multiple products that came across different processing routes are baked in the same oven. Because baking consumes high energy, running an oven while it is only partially occupied causes energy waste. Figure 3 shows a schedule for two products where baking is performed together. In both cases (Figs. 2, 3), the products are internally dependent such that their common tasks must be performed at the same time. This preceding rule is used to arrange products into groups in the flow shop model. Only one product in a group has no predecessor, which means it can be scheduled at any time throughout the production runtime. However, the schedules for the rest of the products in that group depend on it. Table 1 represents simplified production data for one product group which is visualized in Fig. 2.

In reality, many product groups are organized based on their internal dependence. Within a group, each product has an individual bowl time. It indicates the start time difference between a predecessor product and any other product in a group. For one processing stage, there might be multiple alternative machines and employees, from which one should be selected based on availability. In general, for dough rest and cooling, no energy-consuming machines are required and therefore are considered to have the capacity to operate as many products as possible at a time. Similarly, due to having enough space in proofing chambers, it is assumed that the proofing stage has no blockage. Considering the bakers’ practice, ovens can operate multiple products from the same product group at a time (Fig. 3). The rest of the machines can perform a task only from one product.

Figure 4 shows the procedure of optimizing the bakery production schedule. It can be discussed in three distinct segments: data collection, HNFSM, and optimization algorithm. Information about bakery products, machines, and employees is recorded during data collection. Depending on the internal dependence the products are sorted into distinct groups. An initial product sequence, an order of product groups in which they are produced, is transferred to HNFSM. In HNFSM, the processing tasks are allocated among the machines and employees. Here, the actual scheduling, i.e., exact start and end time, machine, or employee to conduct a task is determined. Like other flow shop models, the products that are placed first in the order will get priority in occupying the machines and employees. The following products are scheduled according to machine availability. The makespan and OIDT are calculated as a quality indicator for a schedule, which is considered as a baseline to start improving. The optimization algorithm proposes a new candidate solution vector, which requires to convert into a product group sequence to use for HNFSM. The candidate solution conversion approach is explained later. This procedure is repeated until a certain termination criterion is met.

This section describes the mathematical formulation of the proposed hybrid no-wait flow shop scheduling model. Table 2 defines the notions that are used to describe HNFSM. Ovens at bakeries typically contain multiple compartments that may each be used separately to bake different products. OIDT may exceed the makespan since it calculates the total idle time of all oven compartments. The following equations are used to calculate makespan and OIDT.

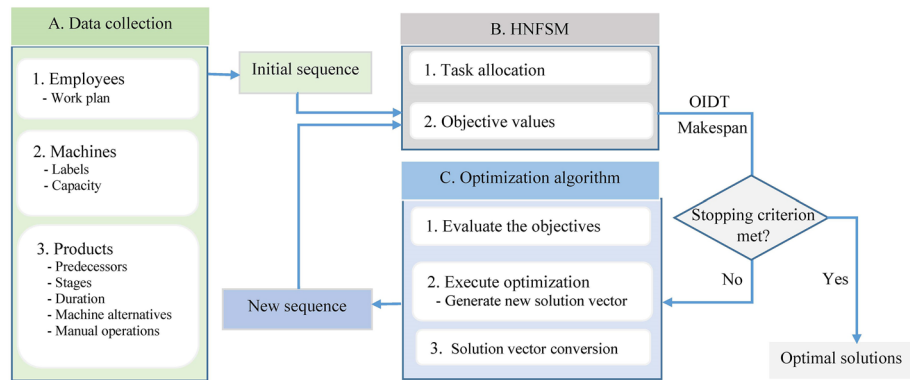


Figure 4. Schematic diagram of bakery production optimization using hybrid no-wait flow shop scheduling model (HNFSM).

Notation	Description
NG	Number of groups of products
n	Number of products in a group
m	Number of machines
e	Number of employees
g	Index of groups; $g = 1, 2, \dots, NG$
p	Index of products in group g ; $p = 1, 2, \dots, n$
k	Index of machines; $k = 1, 2, \dots, m$
l	Index of employees; $l = 1, 2, \dots, e$
s	Index of the processing stage
t	Index of production runtime in minutes
PG	Set of product groups; $G = \{1, 2, \dots, NG\}$
P	Set of products in a group g ; $P = \{1, 2, \dots, n\}$
M	Set of machines; $M = \{1, 2, \dots, m\}$
V	Set of oven compartments; $V \subset M$
U	Set of machines with unlimited capacity; $U \subset M$
E	Set of employees; $E = \{1, 2, \dots, e\}$
$WT_{g,p}$	Time difference between product p and its predecessor in group g
$PT_{g,p,s}$	Processing time at stage s of product p in group g
$PT_{g,p,k}$	Processing time of product p in group g processed by machine k
$ST_{g,p,s}$	Start time for the operation at stage s of product p in group g
$CT_{g,p,s}$	Completion time of stage s of product p in group g
$start_k$	The time when machine k starts its first operation
end_k	The time when machine k finishes its last operation
$start_l$	The time when employee l starts the work
end_l	The time when employee l finishes the work
$O_{g,p,s,k}$	$\begin{cases} 1, & \text{if the product } p \text{ in group } g \text{ is processed on machine } k \text{ at stage } s \\ 0, & \text{if otherwise} \end{cases}$
$O_{g,p,s,l}$	$\begin{cases} 1, & \text{if the product } p \text{ in group } g \text{ is processed by machine } l \text{ at stage } s \\ 0, & \text{if otherwise} \end{cases}$

Table 2. Notations used in hybrid no-wait flow shop scheduling model (HNFSM).

$$Makespan = \max(CT_{g,p,s}) \quad \forall g \in PG, \forall p \in P \quad (M1)$$

$$OIDT = \sum_{k=1}^m \left(end_k - start_k - \sum_{g=1}^{NG} \sum_{p=1}^n PT_{g,p,k} \right) \quad \forall g \in PG, \forall p \in P, \forall k \in V \quad (M2)$$

The HNFSM is described as follows.

$$\text{Min}(\text{Makespan}) \quad (\text{M3})$$

$$\text{Min}(\text{OITD}) \quad (\text{M4})$$

Subject to

$$ST_{g,1,1} \geq 0 \quad \forall g \in PG \quad (\text{M5})$$

$$ST_{g,p,1} = ST_{g,1,1} + WT_{g,p} \quad \forall g \in PG, \forall p \in P \setminus \{1\} \quad (\text{M6})$$

$$PT_{g,p,s} > 0 \quad \forall g \in PG, \forall p \in P, \forall k \in M \quad (\text{M7})$$

$$CT_{g,p,s} = ST_{g,p,s} + PT_{g,p,s} \quad \forall g \in PG, \forall p \in P \quad (\text{M8})$$

$$ST_{g,p,s+1} = CT_{g,p,s} \quad \forall g \in PG, \forall p \in P \quad (\text{M9})$$

$$\sum_{k=1}^m O_{g,p,s,k} \leq 1 \quad \forall g \in PG, \forall p \in P, \forall k \in M \quad (\text{M10})$$

$$\sum_{g=1}^{NG} \sum_{p=1}^n \sum_{t=ST_{g,p,s}}^{CT_{g,p,s}} O_{g,p,s,k} \leq (CT_{g,p,s} - ST_{g,p,s}) \quad \forall g \in PG, \forall p \in P, \forall k \in M \setminus (U \cup V) \quad (\text{M11})$$

$$\sum_{g=1}^{NG} \sum_{p=1}^n \sum_{t=ST_{g,p,s}}^{CT_{g,p,s}} O_{g,p,s,k} \leq n(CT_{g,p,s} - ST_{g,p,s}) \quad \forall g \in PG, \forall p \in P, \forall k \in V \quad (\text{M12})$$

$$\sum_{g=1}^{NG} \sum_{p=1}^n \sum_{t=ST_{g,p,s}}^{CT_{g,p,s}} O_{g,p,s,k} \leq \sum_{g=1}^N n \quad \forall g \in PG, \forall p \in P, \forall k \in U \quad (\text{M13})$$

$$start_l < end_l \quad \forall l \in E \quad (\text{M14})$$

$$\sum_{l=1}^e O_{g,p,s,l} \leq 1 \quad \forall g \in PG, \forall p \in P, \forall l \in E \quad (\text{M15})$$

$$\sum_{g=1}^{NG} \sum_{p=1}^n \sum_{t=ST_{g,p,s}}^{CT_{g,p,s}} O_{g,p,s,l} \leq (CT_{g,p,s} - ST_{g,p,s}) \quad \forall g \in PG, \forall p \in P, \forall l \in E \quad (\text{M16})$$

$$ST_{g,p,s} \geq start_l \quad \forall O_{g,p,s,l} = 1, \forall g \in PG, \forall p \in P, \forall l \in E \quad (\text{M17})$$

$$CT_{g,p,s} \leq end_l \quad \forall O_{g,p,s,l} = 1, \forall g \in PG, \forall p \in P, \forall l \in E \quad (\text{M18})$$

$$\sum_{k=1}^m O_{g,p,s,k} + \sum_{l=1}^e O_{g,p,s,l} \leq 1 \quad \forall g \in PG, \forall p \in P, \forall k \in M, \forall l \in E \quad (\text{M19})$$

The objective functions are shown in Eqs. (M3), (M4). Constraint (M5) states that the start time for the predecessor product of any group can be ≥ 0 . Constraint (M6) defines it for successor products in the group. It includes a time difference between the start time of predecessor and successor products. Constraint (M7) declares that the processing time for any stage must be greater than 0 min. The no-wait condition between two consecutive stages of a product is defined by conditions (M8) and (M9). Constraint (M10) ensures that an operation from a product can occupy only one machine. A machine can only perform one task at a time except for the ovens ($k \in V$) and the machines with unlimited capacity ($k \in U$), as defined by constraint (M11). Constraint (M12) allows ovens to bake multiple products from the same group. According to constraint (M13), machines with unlimited capacity can perform any number of tasks at a time. Condition (M14) validates the shift plan of employees. Constraint (M15) limits the number of employees assigned to a single task. Constraint (M16) restricts the number of tasks assigned to a single employee at any given time. Employee job allocation is limited by constraints (M17) and (M18) to be within their working hours. Condition (M19) states that either an employee or a machine limitation can be occupied for a task. However, in the bakery process, some tasks require no machine and employee, such as dough rest.

Multi-objective optimization. Most real-world optimization problems that scientists and engineers handle routinely are multi-objective problems, where systems demand satisfying more than one parameter. Conventionally, such problems are simplified in two diverse ways: after converting multiple objectives into one by using the linear weighting method and featuring objectives as constraints. These approaches provide an optimized solution to a satisfactory level without handling the complex interrelations between multi-objectives.

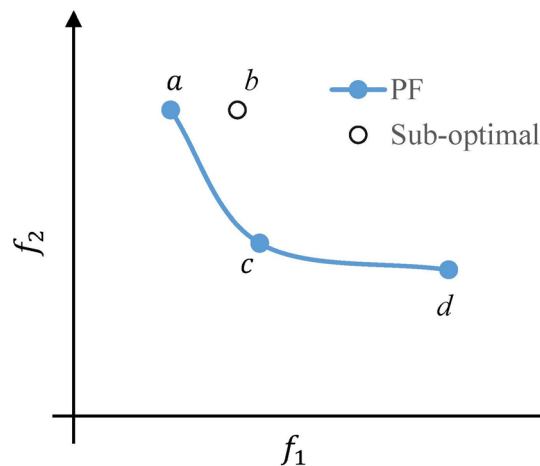


Figure 5. Concept of Pareto dominance for bi-objective functions (f_1 and f_2) optimization minimization problem.

Nonetheless, depending on the type of problems, these approaches have limitations. The former method relies on personal preference when determining the importance of objectives, which has a major impact on the solution. Furthermore, weighting factors might lead the optimizer to a poor solution when solving a problem with a non-convex Pareto front that is unknown beforehand. The latter approach struggles to deal with the high-dimensional, multi-objective optimization problems and is prone to producing suboptimal solutions. In reality, many multi-objective optimization problems do not show continuous solutions in the objective space. Therefore, if objectives are restricted to different ranges, for an optimizer it is challenging to find an optimal solution that meets these constraints. There are many optimal solutions to multi-objective problems with many local minima in a multi-objective space. Following traditional methods, the entire procedure must be repeated many times, each time adjusting the weighting factors or constraints for the objectives to make sure that the obtained solution is not one of these local minima. However, there is no guarantee that a complete set of optimum solutions will be obtained. To address this problem, several multi-objective optimization algorithms have been proposed^{27–33}. Given a decision space χ mapped into $\mathbb{R}$ for q objective functions $f_1 : \chi \rightarrow \mathbb{R}, \dots, f_q : \chi \rightarrow \mathbb{R}$, a multi-objective optimization minimization problem can be stated as follows (Eq. 1).

$$\min f_1(x), \dots, \min f_q(x); x \in \chi \text{ and } q > 1 \quad (1)$$

where $f_1(x), \dots, f_q(x)$ are objective functions such that minimizing one function leads to an increase in others.

Multi-objective optimization, unlike single-objective optimization, generates a collection of optimal solutions by displaying tradeoffs between the objectives in the objective space²⁴. Therefore, an objective vector has q values, such as $[f_1(x), \dots, f_q(x)]$, each of which reflects the extent of the corresponding objective. Figure 5 outlines an objective space for a bi-objective ($q = 2$) problem. The multiple optimal solutions in this space are selected such that they show the best tradeoffs between the objectives, which is defined by Pareto dominance. Pareto dominance is the fundamental of multi-objective optimization algorithms, extensively used to distinguish optimal solutions from suboptimal solutions. To define the Pareto dominance, given two objective vectors, $\vec{a} = [a_1, \dots, a_q]$ and $\vec{b} = [b_1, \dots, b_q]$ and $\vec{a}$ is said to dominate $\vec{b}$ ($\vec{a} \prec \vec{b}$) if and only if $\vec{a}_d \leq \vec{b}_d$ for every $d \in \{1, \dots, q\}$ and $\vec{a}_d < \vec{b}_d$ for at least one of $d \in \{1, \dots, q\}$. In words, $\vec{a}$ dominates $\vec{b}$, if $\vec{a}$ is not worse in any objective and better in at least one objective than $\vec{b}$ ^{34,35}. Figure 5 shows that the objective vector $\vec{a}$ dominates $\vec{b}$ as it improves f_1 while not worsening f_2 . However, considering $\vec{a}$, $\vec{c}$ and $\vec{d}$, no one dominates none and thus together they form the Pareto front (PF).

Optimization algorithms. The concept of Pareto dominance has been used fundamentally in these multi-objective optimization algorithms to find a collection of optimal solutions from a population that progresses over the generations. The strength Pareto evolutionary algorithm (SPEA) is one that was later improved to SPEA2 by eliminating a few weaknesses³². Similarly, the non-dominated sorting genetic algorithm (NSGA) was improved for NSGA-II by reducing the computation complexity using a fast non-dominated sorting approach²⁷. The inclusion of elitism, a feature that preserves the good solutions over generations, in NSGA-II makes it comparable with SPEA2. The third version of generalized differential evolution (GDE3), which originated from the differential evolution algorithm, is relatively a new member of this group³⁶.

Based on the simulation of the social behavior of birds, the particle swarm optimization algorithm was first proposed by Eberhart and Kennedy³⁷. This concept has been used in several studies to develop multi-objective optimization algorithms^{34,38–45}. In an improved particle swarm optimization-based algorithm, known as OMOPSO, Pareto dominance, crowding distance, and mutation operators are included, resulting in highly competitive performance⁴⁴. Later, an extended version, speed-constrained multi-objective particle swarm

Generation (G)	Sequence vector ($\vec{x}_{i,G}$)	Sorted index	Product group sequence
1	[1.22, -1.08, 1.90]	[1-3]	{2, 1, 3}
2	[-0.29, 0.25, 0.80]	[1-3]	{1, 2, 3}
3	[1.06, 0.30, -0.20]	[1-3]	{3, 2, 1}

Table 3. Conversion of a sequence vector to a product group sequence using smallest position value rule.

optimization (SMPSO), was introduced. It is reportedly aimed to adapt particle velocity when it gets higher to generate an effective position in the search space⁴².

However, there is no guarantee that all MOA solutions are truly optimal for an unknown problem. In most previous studies, the effectiveness of multi-objective optimization methods has been demonstrated by solving different mathematical test functions. Solving real-world high-dimensional production scheduling problems are computationally expensive and rarely used as benchmarks to test algorithms. A few studies applied these state-of-art multi-objective algorithms to solve scheduling problems^{46–51}. In Annex A, the algorithms are briefly described.

Solution vector conversion. The optimization candidate solution is a vector consisting of continuous values of size equal to the number of product groups in a problem. In contrast, a solution to HNFSM is a set of discrete numbers, where each number represents one product group. The order of these discrete numbers makes the difference in the final schedule as it implies when each product should be produced. Therefore, a conversion of the algorithmic solution is required. To convert the solution vector into a set of discrete numbers, the smallest position value rule is employed. Table 3 explains the candidate solution conversion procedure for 3 product groups using the smallest position value rule. In this approach, the index of each value in the solution vector is conjugated with a product group. The indexes are sorted by the rule of smallest to the largest value in the vector. The sorted index is used as a solution to the problem. The order of the numbers in the product sequence specifies the order of conjugated product groups in which they should be produced.

Performance indicators. Ye et al.^{52,53} have described the difficulty in achieving effectiveness and efficiency while finding optimized solution to no-wait flow shop scheduling problems. In this study, the performance of an algorithm is evaluated based on obtained Pareto front, called candidate PF, with the true Pareto front (PF*) to a problem. Initially, PF* for a problem is unknown. Once the candidate fronts (PF) for a problem are obtained by algorithms, the PF* is calculated by taking only Pareto optimal solutions from them.

Cardinality. To measure the cardinal quality of optimal solutions, Pareto domination strength is used, which considers the number of optimal and non-optimal solutions in a candidate PF obtained by any algorithm. Pareto domination strength was calculated by using Eq. (2). A higher Pareto domination strength indicates the worst performance of an algorithm.

$$PDS = \frac{|\{a : a \in PF \text{ and } a \notin PF^*\}| - |\{a : a \in PF \text{ and } a \in PF^*\}|}{|PF| \times |PF^*|} \quad (2)$$

where PDS is Pareto domination strength, $|\cdot|$ indicates cardinality of a set, a is an objective vector.

Distribution and spread. The maximum spread of the solutions in a front captures the spread of the solutions in a front using Eq. (3). A higher value for this indicator represents that an algorithm performed better.

$$MSF = \left[\frac{1}{Q} \sum_{q=1}^Q \left[\frac{\min(f_q^{\max}, F_q^{\max}) - \max(f_q^{\min}, F_q^{\min})}{F_q^{\max} - F_q^{\min}} \right]^{27} \right]^{1/2} \quad (3)$$

where MSF is maximum spread of the solutions in a front, Q is the number of objectives, $f_q^{\max}$ and $f_q^{\min}$ are the maximum and minimum values of the q th objective in PF^* , respectively, $F_q^{\max}$ and $F_q^{\min}$ are the maximum and minimum values of the q th objective in the PF provided by the algorithm that is under evaluation.

Convergence. Convergence measures the degree of proximity between PF^* and its approximation, e.g., candidate PF obtained by an algorithm. As a convergence indicator, distance to the Pareto front represents how close the solutions of two fronts are. A higher distance to the Pareto front, calculated by using Eq. (4), indicates an algorithm performed worst.

$$DPF = \frac{1}{|PF^*|} \left(\sum_{a \in PF^*} \min_{b \in PF} Ed(a, b) \right) \quad (4)$$

where DPF is distance to the Pareto front, $Ed(a, b)$ is the Euclidean distance between objective vectors a and b , PF^* is the true Pareto front, and PF is the front obtained by an algorithm that is under evaluation.

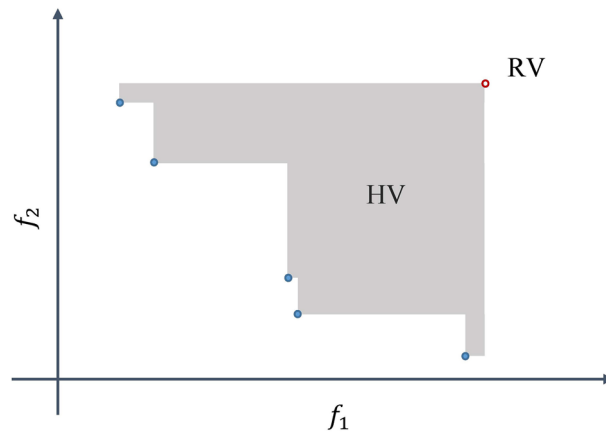


Figure 6. Hypervolume (HV) of one front for two objective functions (f_1 and f_2). Filled circles are solutions in front and empty circle is a reference objective vector (RV).

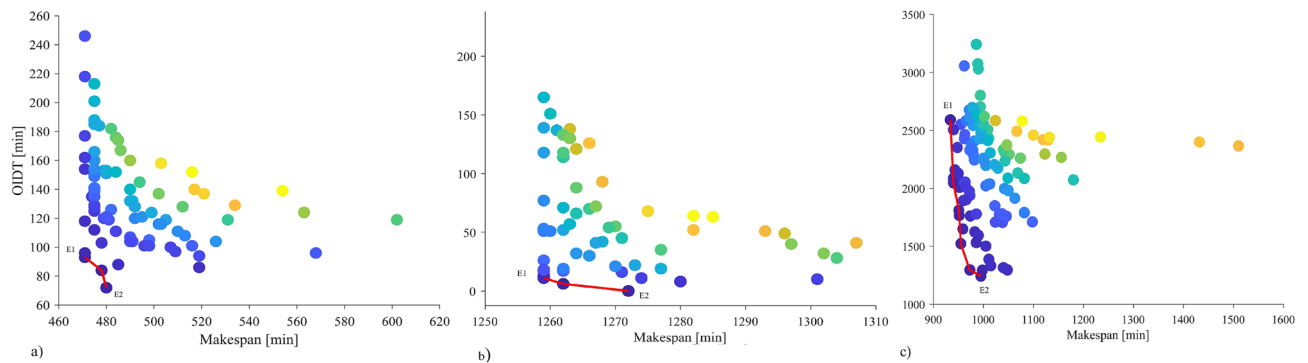


Figure 7. Candidate solutions represented by their quality indicators (circles) obtained by five optimization algorithms and true Pareto front (line) for problem (a) BK15, (b) BK40 and (c) BK50. The color gradient of the circles represents the quality of the solutions, with the darkest blue being the best and pale yellow being the worst.

Convergence and distribution. The hypervolume of the front in objective space describes the convergence and distribution of the solutions obtained by an algorithm. It calculates the volume of the space covered by the solutions of a front and delimited from above by a reference objective vector. It defines the upper limit for each objective in the objective space to consider for calculating hypervolume (Fig. 6) by using Eq. (5). A higher relative hyper volume indicates that an algorithm performed better.

$$HP_{PF} = \lambda_q \left(\bigcup_{a \in PF} [a, RV] \right) \quad (5)$$

where λ_q is the q -dimensional Lebesgue measure, PF is a front obtained by an algorithm that is under test, a is an objective vector and RV is a reference objective vector. In this study, for the two objectives, relative hyper area (Eq. 6) is used to compare the performance of different algorithms.

$$RHA[\%] = \frac{HP_{PF} \times 100}{HP_{PF^*}} \quad (6)$$

where RHA is relative hyper area, HP_{PF} and HP_{PF^*} are hyper volumes for a front obtained by an algorithm that is under evaluation and true Pareto front for a problem, respectively.

Results and discussion

True Pareto front and candidate solutions. Figure 7 shows quality indicators representing candidate solutions in objective space obtained by algorithms. True Pareto front (PF^*) is the line that connects only a set of optimal solutions. PF^* for the instances has only a few solutions, even though many sub-optimal solutions exist nearby. BK15 (Fig. 7a) and BK40 (Fig. 7b) have three Pareto solutions each, but BK50 has seven (Fig. 7c). For BK15 and BK40, the difference between boundary solutions (E1 and E2), which represent extreme tradeoffs between objectives inside PF^* , is insignificant. Boundary solutions for BK15 reveal tradeoffs within a 9 min difference in makespan and 21 min difference in OI DT, while they are 13 min and 11 min differences for BK40,

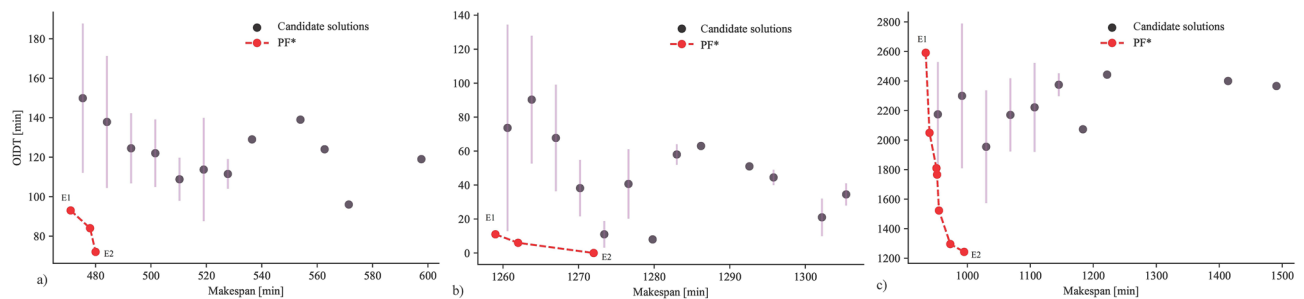


Figure 8. Binned scatter plot of quality indicators representing candidate solutions obtained by five algorithms for instances: (a) BK15, (b) BK40 and (c) BK50. Error bars represent the standard deviation of OIDT.

respectively. In contrast, due to a 61 min rise in makespan, PF* for BK50 conveys a large 1348 min decline in OIDT. Unlike many mathematical function optimizations described in previous studies^{31,36,42}, the solutions in PF* are not continuous in objective space. Furthermore, PF* may have a convex or concave form, which adds to the complexity of solving higher-dimensional problems for optimizers.

Despite a dramatic drop in OIDT over the PF*, BK50 has the highest OIDT (1243 min) at E2. The reason could be that many products in BK50 have predecessors. If multiple products within a group require different specific ovens and the number of products in the group is higher, it is most likely to have higher oven idle time. Candidate solutions that are more densely dispersed in the higher OIDT area support it, with only a few solutions found around E2 (Fig. 7c). In contrast, BK15 has only 3 groups with more than one product and BK40 has no group with multiple products. The PF* of these cases shows a minimum OIDT of 72 min and 0 min, respectively at the E2 point. In addition, their candidate solutions are distributed throughout the objective axis (Fig. 7a,b). BK50 has many groups where the initial stages are processed combinedly to take advantage of machine capacity and save preparation time. Since these products eventually require different baking ovens, finding these ovens available at different time spans leads to a higher OIDT.

Multiple solutions were achieved at the shortest makespan (E1) in all cases, but they were dispersed unevenly, with a few having significantly larger OIDT than the respective Pareto solution (E1). The binned scatter plot (Fig. 8) gives two indications. Firstly, a schedule with minimum makespan does not guarantee to have minimum OIDT. A similar result was observed in previous studies^{5,23}. Therefore, a schedule optimized with a goal to minimize the makespan might be highly inefficient in energy usage. For example, the candidate solutions for BK15 above E1 shows up to 150 min higher OIDT compared to E1 despite having the same makespan, which is even higher for BK40 (231 min). Figure 8 demonstrates that the error bars are pronounced in the region of the lower makespan. It implies that there is a high possibility an optimizer will produce poor solutions around the shortest makespan with a higher energy waste due to OIDT.

Secondly, solutions with a marginal increase in the shortest makespan could result in an acute reduction in OIDT. As a result, a substantial amount of energy can be saved, lowering operational expenses and CO₂ emissions. Because the makespan dominates manufacturing cost, the gain in OIDT is compared to the loss in makespan from the shortest makespan at E1. If any Pareto solution other than E1 offers an intense reduction in OIDT while losing a marginal amount of makespan, the entire manufacturing cost can be reduced even more. For example, for BK50, E2 offers OIDT drop by 8% for each percentage increase in makespan from E1 (Fig. 8c). In other words, E2 is more efficient than E1 since it lowers OIDT by 1348 min while increasing makespan by only 61 min.

Candidate Pareto front. Figure 9 shows the candidate PF for BK15 attained by algorithms. For 50 and 100 iterations, NSGA-II and SPEA2 showed better performance. However, with increasing the iteration size, OMOPSO and GDE3 obtained improved solutions too. In contrast, despite offering a high number of solutions, SMPSO displayed comparatively poor performance. A similar performance was observed for BK50 (not shown). In contrast, for BK40 (not shown), the NSGA-II performed worst compared to OMOPSO and SMPSO. SPEA2 always found only one solution, though it was close to being the optimal solution.

Figure 10 shows the improvement of candidate fronts obtained by algorithms over different iteration sizes for BK50. NSGA-II, SPEA2, and GDE3 improved solution quality remarkably over different iteration sizes (Fig. 10a–c, respectively). However, fronts from OMOPSO and SMPSO were similar, and both displayed poor improvement. All the solutions in PF* for BK50 were obtained by NSGA-II and SPEA2 combinedly, while GDE3 featured a few solutions near PF*. In contrast, no contribution in PF* was observed from MOPSO and SMPSO. Comparable results were obtained for BK15 (not shown). For BK40, GDE3, OMOPSO, and SMPSO obtained Pareto solutions to form PF* and no contribution from NSGA-II and SPEA2 was observed (not shown).

Hecker et al.⁶ used single objective optimization methods to reduce the makespan of BK40, and the results showed that a modified genetic algorithm obtained a minimum makespan of 1261 min 4 times out of 21 runs. Solutions with the best makespan were compared even though multi-objective solutions were found in this study. The best makespan of 1259 min was attained by NSGA-II and SMPSO four times out of four separate runs with varying iteration sizes, while SPEA2, GDE3, and OMOPSO achieved this three times each.

Pareto domination strength and maximum spread of the solutions. Table 4 shows the pareto domination strength and maximum spread of the solution for the algorithms. According to Pareto domination

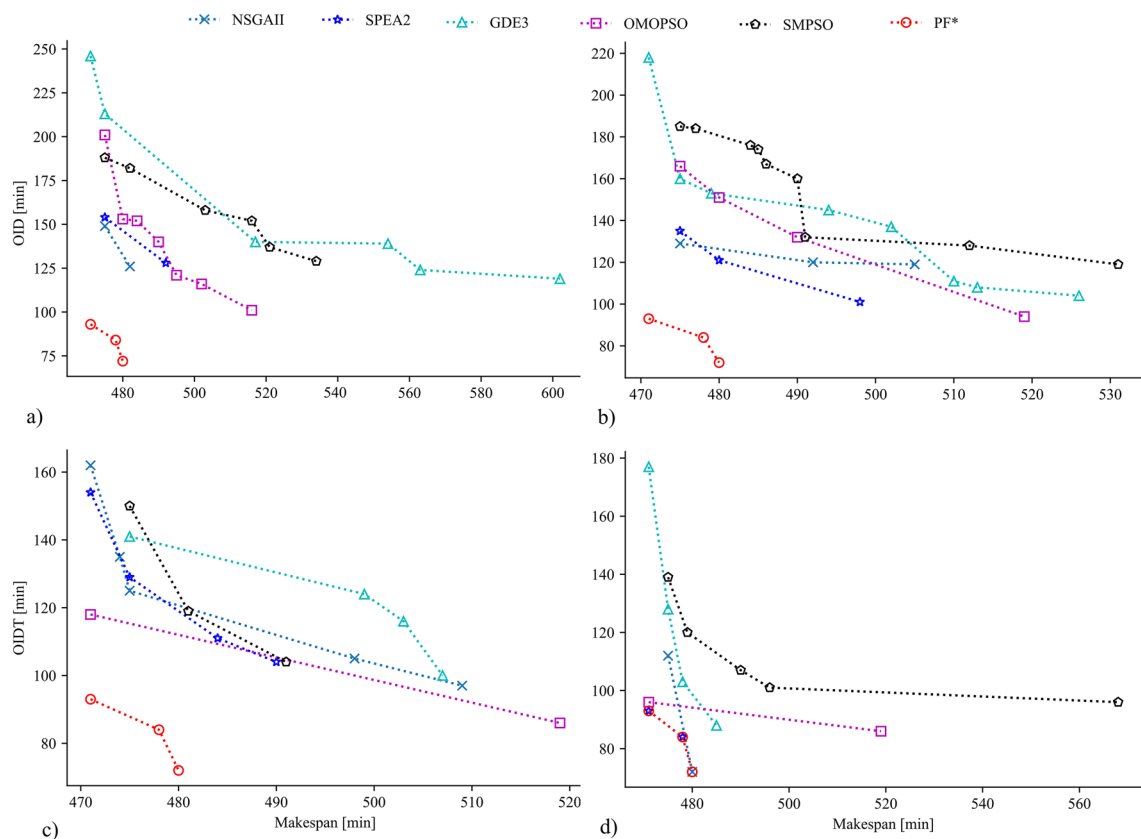


Figure 9. Candidate Pareto front (PF) for BK15 from optimization algorithms with—(a) 50 iterations, (b) 100 iterations, (c) 200 iterations and (d) 300 iterations.

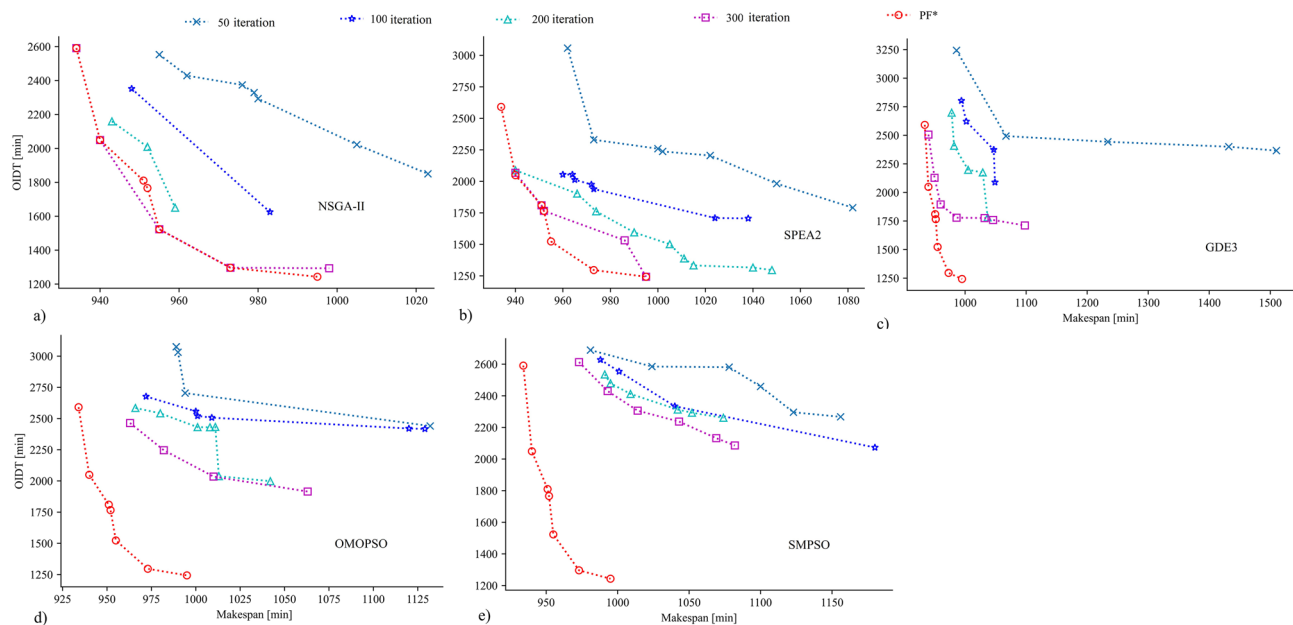


Figure 10. Improvement of candidate Pareto fronts (PF) for BK50 over different iteration size obtained by (a) NSGA-II, (b) SPEA2, (c) GDE3, (d) OMOPSO and (e) SMPSO. A label of 50 iteration indicates the front is obtained by and algorithm with 50 iterations.

Problem	Algorithm	Pareto domination strength				Maximum spread of solutions			
		Iteration				Iteration			
		50	100	200	300	50	100	200	300
BK15	NSGA-II	0.33	0.33	0.20	0.00	1.13	1.84	0.17	0.80
	SPEA2	0.33	0.11	0.17	-0.33	0.97	0.23	0.37	1.00
	GDE3	0.22	0.25	0.33	0.00	0.15	0.13	0.16	0.46
	OMOPSO	0.24	0.17	0.00	0.00	0.10	0.08	0.20	0.51
	SMPSO	0.33	0.33	0.33	0.33	0.44	0.29	0.28	0.06
BK40	NSGA-II	0.17	0.22	0.11	0.11	0.81	0.25	0.44	0.71
	SPEA2	-0.33	-0.33	-0.33	-0.33	0.00	0.00	0.00	0.00
	GDE3	-0.33	-0.11	-0.33	-0.33	0.00	0.61	0.61	0.83
	OMOPSO	0.20	0.14	0.17	0.00	0.26	0.66	0.72	0.42
	SMPSO	0.17	-0.11	0.00	-0.33	0.32	0.76	0.96	1.00
BK50	NSGA-II	0.10	0.14	0.05	-0.09	0.82	1.00	1.00	0.98
	SPEA2	0.10	0.10	0.08	-0.09	0.49	0.78	0.79	1.00
	GDE3	0.14	0.14	0.14	0.10	0.18	0.50	0.66	0.75
	OMOPSO	0.14	0.14	0.14	0.14	0.17	0.48	0.76	0.74
	SMPSO	0.14	0.14	0.10	0.10	0.55	0.66	0.71	0.69

Table 4. Calculated Pareto domination strength and maximum spread of solutions in front for algorithms.

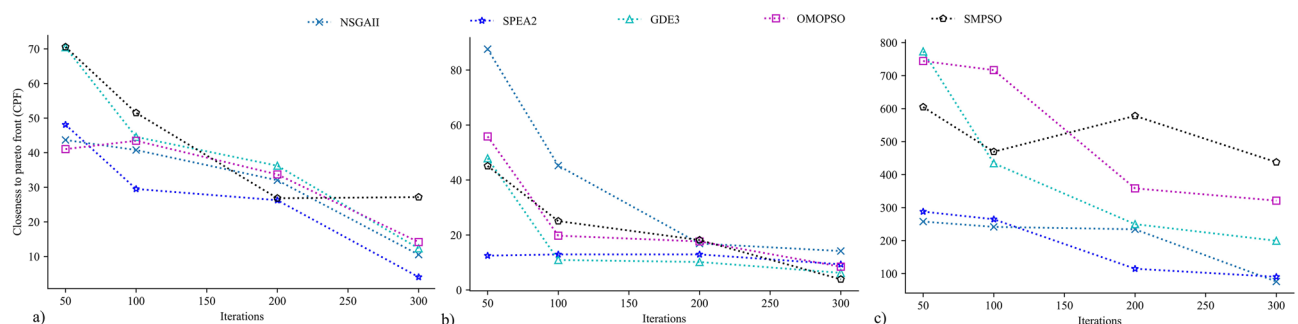


Figure 11. Distance to Pareto front of candidate fronts obtained by algorithms for (a) BK15, (b) BK40 and (c) BK50.

strength, SPEA2, GDE3, and OMOPSO were observed to perform better for solving BK15, with the worst being SMPSO. For BK40, SPEA2 and GDE3 improved solution quality consistently over different iteration sizes. In contrast, NSGA-II performed worst. The Pareto domination strength of NSGA-II and SPEA2 to solve BK50 was better and found no significant difference between them. Similarly, the difference between the Pareto domination strength of GDE3, OMOPSO, and SMPSO is minor and performed worst. According to this performance metric, only SPEA2 showed better performance in all instances.

The maximum spread of the solutions in front measures the distribution and spread of candidate solutions over the PF offered by an algorithm, with a greater number indicating better performance. NSGA-II had higher maximum spread of the solutions in all instances and was found to outperform all other algorithms in this performance metric. The maximum spread of the solutions of GDE3 was equivalent to NSGA-II in most circumstances in terms of problems and iteration sizes. But it had the lowest value for BK40 where only one solution was obtained every time. In contrast, GDE3 had better maximum spread of the solutions for BK40. In most scenarios, GDE3 and OMOPSO was remarkably comparable to each other. With the increasing iteration size to solve BK15, the maximum spread of the solutions of SMPSO decreased. It means that with a short iteration size, it was able to find solutions that had better distribution, but they were mostly suboptimal. In contrast, a large iteration size obtained comparatively better solutions, however, their dispersion was poor. For BK40 and BK50, SMPSO had a modest maximum spread of the solutions.

Distance to Pareto front. The distance to Pareto front for the algorithms is shown in Fig. 11. This performance metric represents how close a candidate's front to PF* is, where a low value indicates better performance. For BK15, SPEA2 exhibited promising improvement over different iteration sizes, while SMPSO was observed to perform worst (Fig. 11a). There is no substantial difference between the distance to Pareto front of NSGA-II, GDE3, and OMOPSO. For BK40, the PF obtained by SPEA2 had the lowest value with minimum

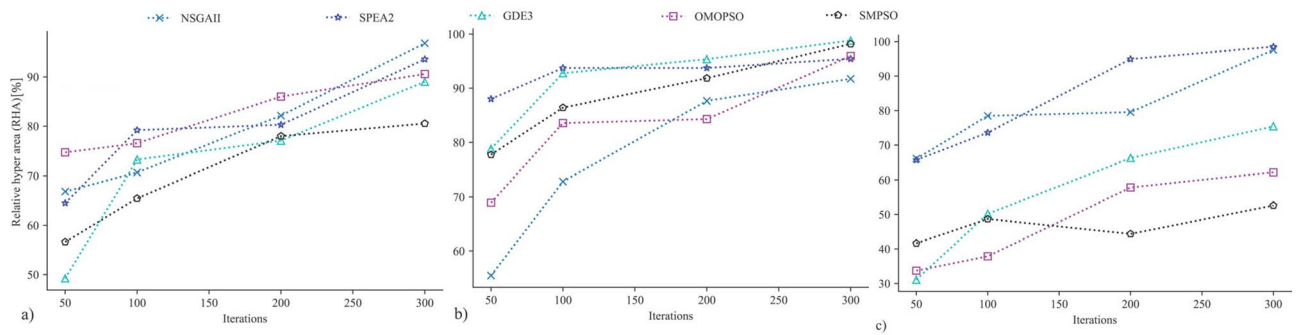


Figure 12. Relative hyper area (RHA) of candidate Pareto fronts (PF) obtained by algorithms for (a) BK15, (b) BK40 and (c) BK50.

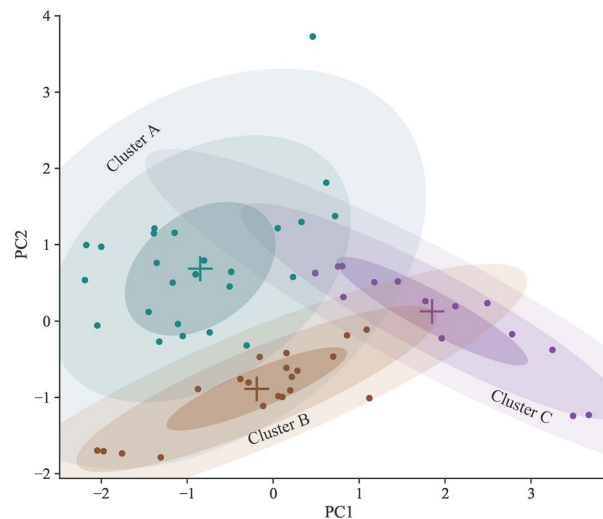


Figure 13. Clusters of algorithms' solutions found by Gaussian mixture model.

iteration (Fig. 11b). However, with increasing iteration sizes, the distance to Pareto front of GDE3, OMOPSO, and SMPSO was comparable to that of SPEA2. According to this performance indicator, NSGA-II had the worst distance to Pareto front with 50 iterations, which sharply improved with increased iteration sizes, yet could not outperform any algorithms. Figure 11c shows the distance to Pareto front of algorithms for BK50, where NSGA-II and SPEA2 outperformed GDE3, OMOPSO, and SMPSO. The values of BK50 distinguished algorithms' performance at every iteration which was not prominent for BK15 (Fig. 11a) and BK40 (Fig. 11b).

Relative hyper area. Figure 12 presents the relative hyper area of algorithms. It measures convergence and distribution of algorithms with a higher value indicating better performance. NSGA-II had higher value for BK15 and BK50, while for BK40 it performed worst, and SPEA2 had higher relative hyper area for all the cases. GDE3 and OMOPSO had moderate relative hyper area for all instances. In contrast, SMPSO showed the lowest relative hyper area for BK15 and BK50, and higher for BK40.

The distance to Pareto front and the relative hyper area for BK15 and BK40 illustrate the significant improvement in solutions' quality over different iteration sizes for algorithms (Figs. 11, 12). With the higher iteration size, the performance difference between algorithms was found to be minimum. Only NSGA-II, SPEA2, and GDE3 were able to follow this trend in BK50, while OMOPSO and SMPSO fell behind. One reason could be that there are more local minima in the solution space of BK50 compared to that of BK15 and BK40. Many suboptimal solutions exist for BK50 with higher OIOT with a small difference in makespan (Fig. 8c). Additionally, BK50 has a higher dimension—maximum product groups—to optimize.

Performance evaluation of algorithms. Performance metrics explain a specific feature of solution quality. The solutions, however, can be categorized into different quality levels using a clustering approach. The frequency with which an algorithm produces good or poor-quality solutions is a measure of its efficiency. Initially, the performance metrics for all instances are used to perform principal component analysis (PCA)⁵⁴. Two principal components (PC1 and PC2) with higher variances were taken to perform a Gaussian mixture model for clustering²². Figure 13 shows three clusters. The clusters' solutions were identified using the associated labels, which refer to instances, algorithms, and iteration sizes such as BK15, NSGA-II, and 50, respectively. The quality

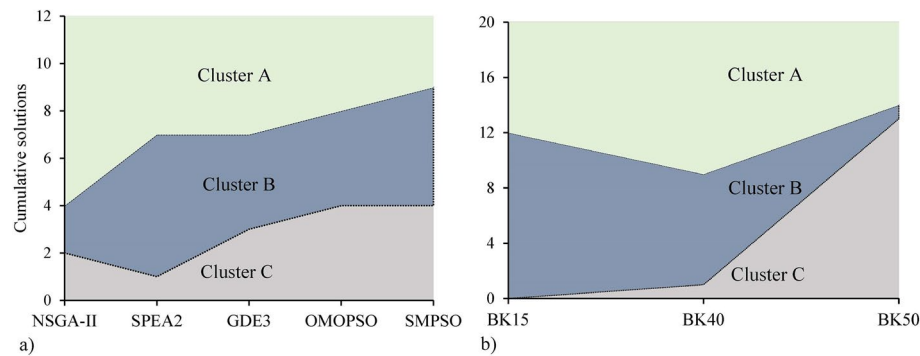


Figure 14. Distribution of obtained solutions among clusters with respect to: (a) algorithms (b) problems.

of different clusters were determined based on corresponding performance metrics where Cluster A represents better performances, and Cluster B and Cluster C show moderate and worst performances, respectively.

Figure 14 represents the distribution of obtained solutions into three clusters. Cluster A has 25 high-quality solutions, Cluster B has 21, and Cluster C has 14 low-quality solutions for the instances. NSGA-II had the highest number of solutions in Cluster A, followed by SPEA2 and GDE3 (Fig. 14a). In contrast, OMOPSO and SMPSO have the lowest number of solutions in this cluster. NSGA-II, with only four moderate and worst solutions, outperformed all other algorithms. SPEA2, with the highest number of moderate and lowest worst solutions, followed NSGA-II. In terms of distribution of obtained solutions among clusters, GDE3 performed slightly better than OMOPSO. In comparison to NSGA-II and SPEA2, SMPSO has the lowest solutions in Cluster A and the highest in Cluster C, indicating worse performance. For BK15, a large number of obtained solutions were moderate, with no worst solution (Fig. 14b). In contrast, for BK40, the majority of solutions fell into cluster A, emphasizing a problem that is comparatively easy to solve. BK50 revealed a considerable rise in the difficulty of obtaining moderate and better solutions, with Cluster C accounting for 65% of all solutions. Three of the six solutions in Cluster A were achieved by NSGA-II, two by SPEA2, and one by GDE3. In Cluster B, there is only one solution for BK50, which was obtained by SPEA2. In contrast, all the solutions from OMOPSO and SMPSO are in Cluster C. With only one Cluster C solution for BK50, NSGA-II and SPEA2 displayed consistently better performance. According to the cluster analysis, NSGA-II outperformed all other algorithms, followed by SPEA2. GDE3 performed better than OMOPSO and SMPSO, but OMOPSO and SMPSO showed no notable difference in performance.

The comparison of computation time of algorithms is performed with 50 iterations for the instances. Although OMOPSO needed the shortest calculation time (12 min, 58 min, 324 min for BK15, BK40 and BK50, respectively), the difference between it and other methods is insignificant. It took roughly the same amount of time for NSGA-II and SPEA2 in each case—13 min, 62 min, and 342 min, respectively. GDE3 showed slightly lower computation time with instances taking 12 min, 61 min, and 337 min, respectively. SMPSO, in contrast, had the longest computing time for every instance (13 min, 67 min, and 360 min, respectively). In comparison to OMOPSO, the extension to SMPSO appears to have triggered slightly high computing time as velocity constraints are applied to each iteration and dimension of the problem.

The current study shows that production planning using a flow shop model is feasible in practice when considering the actual resource limitations in bakeries. Along with makespan, minimizing the oven idle time also offers the potential to substantially lower manufacturing costs. To improve the current state of production efficiency in real cases from bakeries, multi-objective optimization algorithms were integrated with hybrid no-wait flow shop model. Among them, NSGA-II performed better in solving problems of various dimensions. Moreover, when multiple products share a predecessor, the increased oven idle time results in energy loss. Therefore, wherever possible, it is suggested to keep the processing route for a product separate from other products. Six bakery production datasets from Denmark were used by Babor et al.⁵⁵ to increase the production efficiency. The results revealed that NSGA-II performed efficiently to reduce makespan by up to 12% and oven idle time by up to 61%. Particle swarm optimization was used in a study⁵ to obtain the best planning for a bakery's production in Spain. The optimum solution, according to the results, minimized the makespan by 29% and the oven idle time by 8%.

Conclusions and future works

In this paper, three production optimization problems from small and medium-sized bakeries were investigated. The objectives of optimization were to minimize simultaneously makespan and oven idle time (OIDT). A hybrid no-wait flow shop scheduling model with all constraints encountered in practice was implemented to simulate the bakery schedule. The optimum schedules were found using five multi-objective optimization algorithms: non-dominated sorting genetic algorithm (NSGA-II), strength Pareto evolutionary algorithm (SPEA2), generalized differential evolution (GDE3), improved multi-objective particle swarm optimization (OMOPSO), and speed-constrained multi-objective particle swarm optimization (SMPSO). To compare the efficiency of the algorithms, each problem was solved with different iteration sizes.

The computational results revealed that the shape of a true Pareto front is determined by the characteristics of the problems, such as the number of items, product interdependency, and alternative machinery. Although makespan has the most influence on production expenditure, it was observed that a substantial reduction in

OIDT is possible. Many solutions with the shortest makespan had higher OIDT (up to 231 min) that showed significant energy waste and CO₂ emissions. Therefore, with the same makespan, multi-objective algorithms can provide solutions with reduced energy waste. Furthermore, many Pareto solutions, aside from the one with the shortest makespan, provide better tradeoffs between makespan and OIDT. It means that by losing a very marginal amount in makespan, some solutions offer a substantial reduction in OIDT. BK50 showed an additional 1348 min of oven idle time can be reduced if the makespan is increased by 61 min. Therefore, the overall production expenditure can be significantly minimized. Product group formulation may influence OIDT. In the best-case scenario, for BK40 with no predecessor in any group, a schedule with 0 min OIDT is possible. However, because many products have a few combined initial processing stages, for BK50 the lowest possible OIDT is 1243 min, resulting in significant energy loss.

NSGA-II outperformed other algorithms by obtaining a smaller number of poor solutions and a high number of better solutions. SPEA2 followed NSGA-II by delivering promising solutions. GDE3 performed slightly better than OMOPSO and SMPSO. The performance of OMOPSO and SMPSO was poor to solve the instances and no significant difference between them was observed. However, OMOPSO had the lowest computation time while SMPSO had approximately 11% higher computation time due to the addition of velocity constraints.

The deterministic duration of the processing tasks and the absence of machine maintenance or failure assumed in this study may not reflect many realistic production problems. Based on prior relevant studies^{56,57}, the effects of non-deterministic processing duration and machine disturbances on the production efficiency of bakeries could be an interesting subject for future research.

Data availability

For this study, production data from bakeries in Europe were used. BK40 was collected and analyzed by Hecker et al.⁶, whereas BK15 and BK50 are publicly accessible²⁵. The production data are available from the corresponding author on reasonable request.

Appendix

Multi-objective optimization algorithms

Non-dominated sorting genetic algorithm (NSGA-II). The following is the description of the NSGA-II (Fig. 15) proposed by Deb et al.²⁷.

Create a random population of size NP . Each individual in the population is a candidate solution vector to a problem. Assess the individuals in the population using HNFSM by calculating objective values.

1. The objective values are used to build fitness vectors. The population is sorted into distinct rankings using the fast non-dominated sorting strategy. Using the Pareto dominance operator, each individual's fitness is compared to that of others. The rank of an individual is determined by the number of other individuals who dominate it, which is known as the sum of domination. If the sum of domination for an individual is 0, it is called a non-dominated solution or Pareto optimal solution. Individuals with a higher sum of domination have a suboptimal solution to the problem.
2. A binary tournament selection process is used to choose two parents from the existing population for creating two offspring. In the binary tournament selection process, four individuals from the current population are picked and the one with the best rank is chosen as one of the parents. The same procedure is followed to complete the parents' poll to perform crossover and mutation. It is repeated to create new offspring of NP size.
3. Evaluate offspring to get the corresponding fitness vectors. The offspring and parent population are combined. During this phase, the population doubles in size.
4. The combined population is sorted into different ranks using the fast non-dominated sorting approach.
5. To select the best population of NP size, the individuals with the best rank are chosen first. If the best rank does not have enough individuals to fill all the empty slots of the best population, the individuals of subsequent ranks are chosen. If a rank has more individuals than empty slots, the crowding distance operator is used to select individuals from the less crowded part of the objective space. The crowding distance is set to infinity for border solutions of a rank to give preference over others. All the solutions of the rank are sorted in descending order of crowding distance. Individuals with a higher crowding distance fill the empty slot first until the best population size reaches n .
6. Employ crossover and mutation operators to produce NP offspring from the best population.
7. Repeat steps 4–7 until the termination criterion is met.

A.2: Strength Pareto evolutionary algorithm (SPEA2). The process flowchart for SPEA2 is shown in Fig. 16. The brief of SPEA2³² is as follows:

1. Generate an initial population of size N where each individual is a candidate solution vector for a problem. Create an empty external archive. Set the size limit for external archives to EA . In this archive, the best individuals are stored.
2. Evaluate the individuals to get fitness values. The fitness of the individuals in the population is calculated. Initially, a strength value for each individual is calculated using Eq. (A1).

$$S(i) = |\{j | j \in PO_G + AP_G \wedge i > j\}| \quad (\text{A1})$$

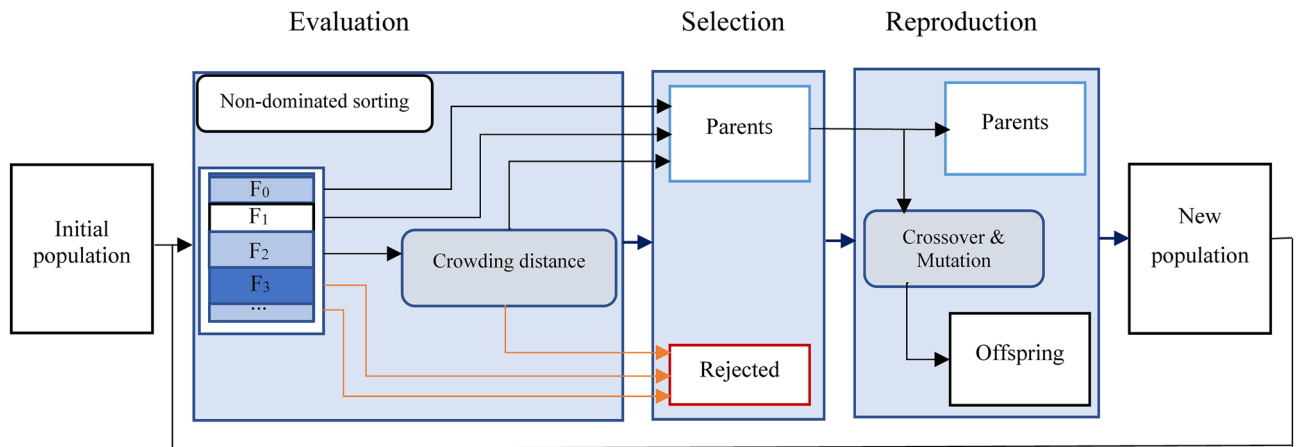


Figure 15. An evaluation procedure of the NSGA-II⁵⁵. Non-dominated sorting divides the population into different ranks ($F_0, F_1, F_2, \dots$). Individuals from the same optimal front are maintained in one rank.

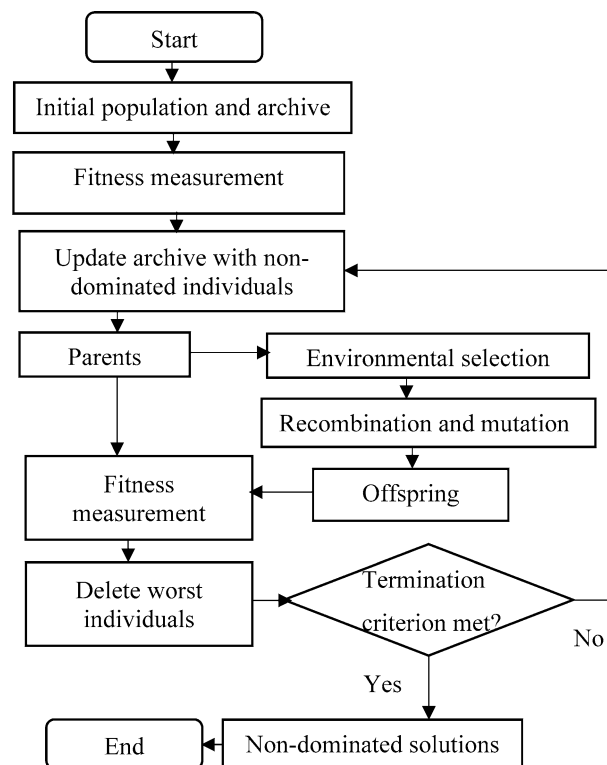


Figure 16. Flowchart of strength Pareto evolutionary algorithm (SPEA2).

where $|\cdot|$ indicates the cardinality of a set, $+$ sign is for multiset union, $>$ symbolizes the Pareto dominance relation, PO_G is the population and AP_G is archive size at generation G . Therefore, $S(i)$ represents the number of other individuals (j) in the population and archive that are dominated by an individual i .

The raw fitness of individual i is defined by Eq. (A2).

$$R(i) = \sum_{j \in PO_G + AP_G, j > i} S(j) \quad (\text{A2})$$

Equation (3) implies that the raw fitness of an individual is the sum of the strength of its dominators. In the next stage, a density estimation approach is employed. For that purpose, k th nearest neighbor method is adapted. The distances between i and all other individuals (j) in objective space are calculated. The distance list is sorted in increasing order. The k th element gives the distance sought σ_i^k . Equation (A3) defines the density calculation for an individual.

$$D(i) = \frac{1}{\sigma_i^k + 2} \quad (\text{A3})$$

where $k = \sqrt{NP + EA}$, NP and EA are the size of the total population and archive, respectively. Finally, the fitness of an individual i can be stated as follows:

$$F(i) = R(i) + D(i) \quad (\text{A4})$$

3. Take all non-dominated individuals to archive. If the size of the archive exceeds the limit, reduce it by using a truncation operator that prevents boundary solutions from being removed. If the size is less than the limit, fill it with dominated individuals.
4. Employ binary tournament selection with replacement to obtain offspring. Initially, all individuals are compared based on Pareto dominance. Rank the individuals depending on their domination level. Afterward, estimate their density information within the corresponding rank. It represents the sum of distances between the two closest individuals along with each objective. Based on these two sorting approaches, the parent selection is performed. Apply recombination and mutation operators to keep high diversity among the population.
5. Combine offspring, parent population, and archived individuals. Delete the worst 50% of combined population based on their fitness values (as shown in step 2).
6. Continue with steps 3–5 until a stopping criterion is met.

A.3: Generalized differential evolution (GDE3). The differential evolution algorithm was first introduced by Storn and Price⁵⁸. Like all other evolutionary algorithms, it has a random initial population, which is improved over the generations. It features cross-over, mutation, and selection operators to improve the solution. The selection rule is one of the key differences compared to other evolutionary algorithms. It is the process to decide whether a new individual should replace one from the population to generate efficient individuals for the next generation. The decision is taken based on some constraints that are regulated by crossover constant and differential variation between two individuals. Later, differential evolution algorithm was extended to generalized differential evolution (GDE) for multi-objective optimization problems by modifying the selection rule⁵⁹. The optimization procedure of GDE3 is presented in Fig. 17. GDE3, an improved version, can be described as follows³⁶.

1. Initialize population of size NP ($NP \geq 4$), amplification constant for differential variation, $F \in (0, 1+]$, crossover constant, $CR \in [0, 1]$, dimensions or parameters of the problem, D , the maximum generation G_{max} , and a constant $A = 0$. The solution vector is $\vec{x}_{i,G}$ for an individual i at G generation where $i = \{1, 2, \dots, NP\}$, and $G = \{1, 2, \dots, G_{max}\}$. The value of d dimension in i individual at G generation is indicated by $x_{d,i,G}$, where $d = \{1, 2, \dots, D\}$.
2. Mutate and recombine each individual in the population. For an individual i , choose three different individuals randomly R_1, R_2, R_3 where $R_1 \neq R_2 \neq R_3 \neq i$ and $R_1, R_2, R_3, i \in \{1, 2, \dots, NP\}$. Choose random parameter d_{rand} where $d_{rand} \in \{1, 2, \dots, D\}$. For each dimension (d), the following procedure is applied to $\vec{x}_{i,G}$ to get a new individual, which is known as a trial vector ($\vec{u}_{i,G}$).

$$u_{d,i,G} = \begin{cases} x_{d,R_3,G} + F \times (x_{d,R_1,G} - x_{d,R_2,G}) & \text{if } r < CR \text{ or } d = d_{rand} \\ x_{d,i,G} & \text{otherwise} \end{cases} \quad (\text{A5})$$

where r is a random value between 0 and 1 and $d_{rand} \in \{1, 2, \dots, D\}$

3. Decide whether the trial vector should become a member of generation $G + 1$.

$$\vec{x}_{i,G+1} = \begin{cases} \vec{u}_{i,G} & \text{if } \vec{u}_{i,G} \preceq_c \vec{x}_{i,G} \\ \vec{x}_{i,G} & \text{otherwise} \end{cases} \quad (\text{A6})$$

where the symbol $\preceq_c$ is constraint domination. To define it, $\vec{u}_{i,G}$ constraint dominates $\vec{x}_{i,G}$ if any of the following conditions is true⁶⁰:

$\vec{u}_{i,G}$ is feasible and $\vec{x}_{i,G}$ is not.
 $\vec{u}_{i,G}$ and $\vec{x}_{i,G}$ are infeasible and $\vec{u}_{i,G}$ dominates $\vec{x}_{i,G}$ in constraint function space.
 $\vec{u}_{i,G}$ and $\vec{x}_{i,G}$ are feasible and $\vec{u}_{i,G}$ dominates $\vec{x}_{i,G}$ in objective space.
 Set the following conditions:

$$A = A + 1 \quad \vec{x}_{NP+A,G+1} = \vec{u}_{i,G} \quad \text{if} \quad \begin{cases} \forall d: C_d(\vec{u}_{i,G}) \leq 0 \\ \text{and} \\ \vec{x}_{i,G+1} == \vec{x}_{i,G} \\ \text{and} \\ \vec{x}_{i,G} \nprec \vec{u}_{i,G} \end{cases} \quad (\text{A7})$$

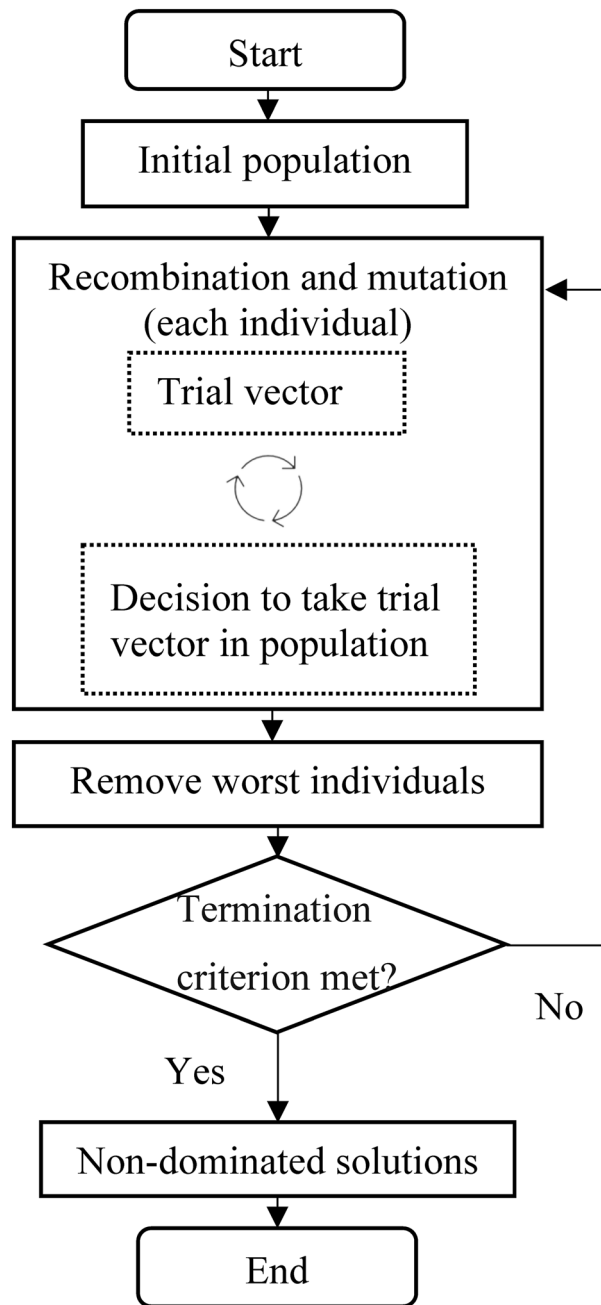


Figure 17. Optimization procedure for generalized differential evolution (GDE3).

where $C_d(\vec{u}_{i,G})$ indicates constraint associated with d th dimension.

4. Repeat steps 2–3 NP times to complete mutation and recombination of the population of the generation G .
5. Select individuals $(\vec{x})$ that meet the following condition:

$$\vec{x} \in \{\vec{x}_{1,G+1}, \vec{x}_{2,G+1}, \dots, \vec{x}_{NP+A,G+1}\} : \begin{cases} \forall i \ \vec{x} \nlessdot_c \vec{x}_{i,G+1} \\ \text{and} \\ \forall (\vec{x}_{i,G+1} : \vec{x}_{1,G+1} \nlessdot_c \vec{x}) \quad CD(\vec{x}) \leq CD(\vec{x}_{i,G+1}) \end{cases}$$

(A8)

where CD is crowding distance that measures the crowdedness of a vector in a non-dominated set.

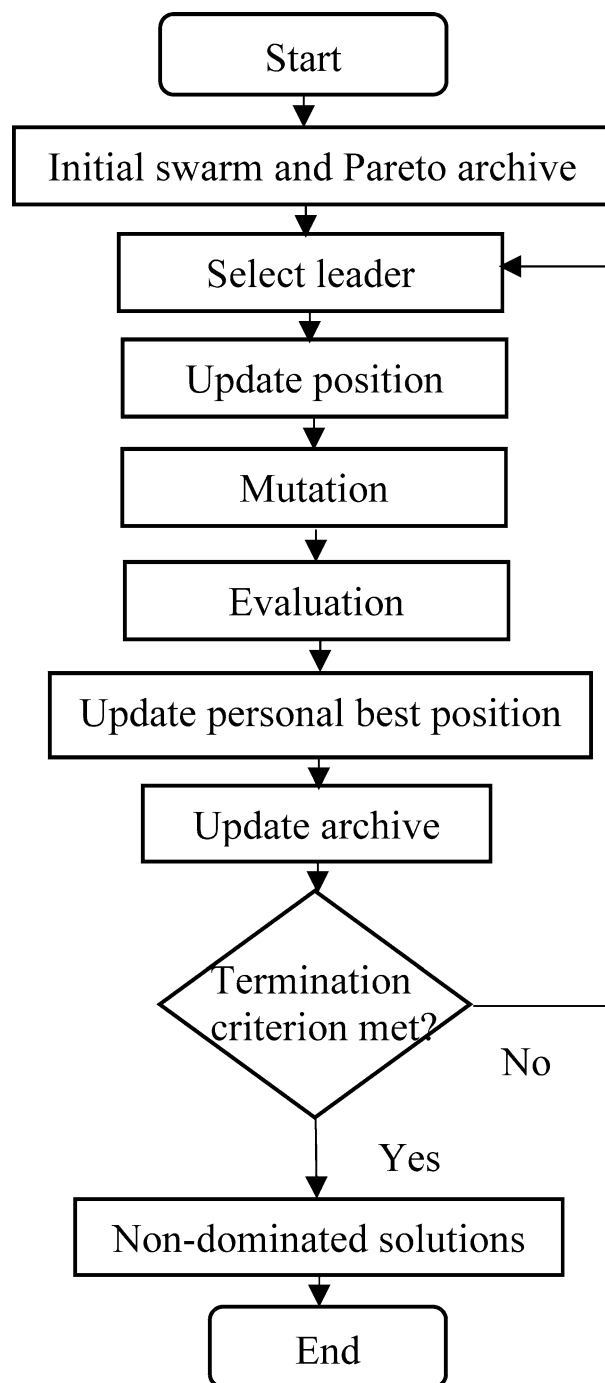


Figure 18. Flowchart of improved multi-objective particle swarm optimization algorithm (OMOPSO) to obtain non-dominated solutions.

6. Remove the individuals in $\vec{x}$ from population. Set $A = A - 1$ and repeat step 5 while $A > 0$.
7. Increase the generation from G to $G + 1$.
8. Repeat the steps 2 – 7 while $G \leq G_{max}$.

A.4: Improved multi-objective particle swarm optimization (OMOPSO). Figure 18 shows the flowchart for OMOPSO. The following is the description of OMOPSO proposed by Sierra and Coello⁴⁴.

1. Initialize a swarm where each particle in the swarm is a candidate solution vector to solve a problem. Evaluate them and initialize the best position and velocity of the particles. Set the size for leaders and initialize leaders

Parameters	NSGA-II, SPEA2	GDE3	OMOPSO	SMPSO
Population size	50	50	50	50
Mutations	Polynomial	–	Uniform, non-uniform	Polynomial mutation
Mutation rate	$\frac{1}{\text{Productgroups}}$	–	$\frac{1}{\text{Productgroups}}$	$\frac{1}{\text{Productgroups}}$
Crossover	Simulated binary	Differential evolution	–	–
Crossover rate	1	1	–	–
Selection	Binary tournament	Differential evolution	–	–
Other parameters		CR = 1 F = 0.4	C1 = random (1.5, 2) C2 = random (1.5, 2) W = random (0.1, 0.5)	C1 = random (1, 2.5) C2 = random (1, 2.5) W = 0.2

Table 5. Parameters setting for the algorithms.

- from the existing swarm. Save the leaders in the Pareto archive. Initialize generations, $G = \{1, 2, \dots, G_{max}\}$. The crowdedness of the leaders is calculated.
- For each particle select a leader through a binary tournament. The selection criterion is the crowding distances where a leader with a higher crowding distance is chosen.
 - Update the position of the particle to a new position using Eqs. (A9), (A10).

$$\vec{v}_{i,G} = W \times \vec{v}_{i,G-1} + C_1 \times r_1 \times (\vec{x}_{i,best} - \vec{x}_{i,G-1}) + C_2 \times r_2 \times (\vec{L}_h - \vec{x}_{i,G-1}) \quad (\text{A9})$$

$$\vec{x}_{i,G} = \vec{x}_{i,G-1} + \vec{v}_{i,G} \quad (\text{A10})$$

- where i indicates one particle, $\vec{v}$ is the velocity, W is the inertia weight, C_1 and C_2 are velocity control parameters, r_1 and r_2 are random numbers between 0 and 1, $\vec{x}_{i,best}$ and $\vec{x}_{i,G}$ are particle's best position and current position at generation G , respectively, $\vec{L}_h$ is a position vector of the selected leader of the h index from the Pareto archive.
- Divide swarm into three parts to employ distinct mutation treatments: no mutation, uniform mutation, and non-uniform mutation.
 - The particles are evaluated. The personal best position (x_{best}) for each particle is updated by comparing the current and personal best fitness.
 - Update leader set in archive by including non-dominated Pareto solutions and removing dominated solutions. Calculate the crowding distance of the leaders. Eliminate leaders based on crowding distance if the size of the archive exceeds the limit. Increase the generation from G to $G + 1$.
 - Repeat the process (steps 2–6) until G reaches G_{max} . Save the Pareto archive as the set of optimal solutions to the problem.

A.5: Speed-constrained multi-objective particle swarm optimization (SMPSO). SMPSO, an extended version of OMOPSO, was proposed by Nebro et al.⁴². In this proposal, the values for velocity parameters in Eq. (10), C_1 and C_2 are controlled by using following constriction coefficient (χ) calculated by Eq. (A11).

$$\chi = \frac{2}{2 - \varphi - \sqrt{\varphi^2 - 4\varphi}} \quad (\text{A11})$$

$$\text{where } \varphi = \begin{cases} C_1 + C_2 & \text{if } C_1 + C_2 > 4 \\ 1 & \text{otherwise} \end{cases}$$

The velocity of the particle in each parameter d is bounded using the following velocity constriction equations (Eqs. A12, A13):

$$v_{i,d,G} = \begin{cases} \text{delta}_d & \text{if } v_{i,d,G} > \text{delta}_d \\ -\text{delta}_d & \text{if } v_{i,d,G} \leq -\text{delta}_d \\ v_{i,d,G} & \text{otherwise} \end{cases} \quad (\text{A12})$$

$$\text{delta}_d = \frac{UB_d - LB_d}{2} \quad (\text{A13})$$

where UB_d , and LB_d are upper bound and lower bound of the parameter d .

To summarize the modifications in SMPSO from OMOPSO, for each particle, the velocity is calculated by Eq. (A10), which is then multiplied by the constriction coefficient (χ) (Eq. A11). The resulting value for each parameter is constrained by Eqs. (A12), (A13). The rest of the procedure is the same as OMOPSO. Table 5 shows the parameters setting of the algorithms.

Received: 9 August 2022; Accepted: 21 December 2022

Published online: 05 January 2023

References

- Babor, M. *et al.* *Automation Science and Technology* (Food2Multimedia GmbH, Radbruch, 2021).
- Zentralverband des Deutschen Bäckerhandwerks e. V. Wirtschaftsfaktor Bäckerhandwerk. (2022).
- Gonzalez, T. & Sahni, S. Flowshop and jobshop schedules: Complexity and approximation. *Oper. Res.* **26**, 36–52 (1978).
- Liang, Z., Zhong, P., Liu, M., Zhang, C. & Zhang, Z. A computational efficient optimization of flow shop scheduling problems. *Sci. Rep.* **12**, 845 (2022).
- Babor, M., Senge, J., Rosell, C. M., Rodrigo, D. & Hitzmann, B. Optimization of no-wait flowshop scheduling problem in bakery production with modified PSO. *NEH SA. Process.* **9**, 2044 (2021).
- Hecker, F. T., Stanke, M., Becker, T. & Hitzmann, B. Application of a modified GA, ACO and a random search procedure to solve the production scheduling of a case study bakery. *Expert Syst. Appl.* **41**, 5882–5891 (2014).
- Swangnop, S., Duangdee, T. & Duangdee, J. Design of production planning process for bakery manufacturer. In *2019 IEEE 6th International Conference on Industrial Engineering and Applications (ICIEA)* 178–182 (2019). <https://doi.org/10.1109/IEA.2019.8714851>.
- Wang, S., Wang, X., Chu, F. & Yu, J. An energy-efficient two-stage hybrid flow shop scheduling problem in a glass production. *Int. J. Prod. Res.* **58**, 2283–2314 (2020).
- Liu, L., Chang, Z. & Song, S. Optimization of a molten iron scheduling problem with uncertain processing time using variable neighborhood search algorithm. *Sci. Rep.* **12**, 7303 (2022).
- Fathollahi-Fard, A. M., Woodward, L. & Akhrif, O. Sustainable distributed permutation flow-shop scheduling model based on a triple bottom line concept. *J. Ind. Inf. Integr.* **24**, 100233 (2021).
- Dziuranski, P. *et al.* Solving the multi-objective flexible job-shop scheduling problem with alternative recipes for a chemical production process. In *Applications of evolutionary computation* (eds Kaufmann, P. & Castillo, P. A.) 33–48 (Springer, 2019). https://doi.org/10.1007/978-3-030-16692-2_3.
- Du, J., Zhang, Z., Li, M., Guo, J. & Zhu, K. Optimal scheduling of integrated energy system based on improved grey wolf optimization algorithm. *Sci. Rep.* **12**, 7095 (2022).
- Ala, A., Alsaadi, F. E., Ahmadi, M. & Mirjalili, S. Optimization of an appointment scheduling problem for healthcare systems based on the quality of fairness service using whale optimization algorithm and NSGA-II. *Sci. Rep.* **11**, 19816 (2021).
- Ikedo, K., Nakamura, Y. & Humble, T. S. Application of quantum annealing to nurse scheduling problem. *Sci. Rep.* **9**, 12837 (2019).
- Valdano, E., Poletto, C., Boëlle, P.-Y. & Colizza, V. Reorganization of nurse scheduling reduces the risk of healthcare associated infections. *Sci. Rep.* **11**, 7393 (2021).
- Wang, S., Wang, X. & Yu, L. Two-stage no-wait hybrid flow-shop scheduling with sequence-dependent setup times. *Int. J. Syst. Sci. Oper. Logist.* **7**, 291–307 (2020).
- Yuksel, D., Tasgetiren, M. F., Kandiller, L. & Pan, Q. -K. Metaheuristics for energy-efficient no-wait flowshops: A trade-off between makespan and total energy consumption. In *2020 IEEE Congress on Evolutionary Computation (CEC)* 1–8 (IEEE, 2020). doi:<https://doi.org/10.1109/CEC48606.2020.9185554>.
- Zhang, F., Bai, J., Yang, D. & Wang, Q. Digital twin data-driven proactive job-shop scheduling strategy towards asymmetric manufacturing execution decision. *Sci. Rep.* **12**, 1546 (2022).
- Zhou, L. *et al.* Production and operations management for intelligent manufacturing: A systematic literature review. *Int. J. Prod. Res.* **60**, 808–846 (2022).
- Huber, J. & Stuckenschmidt, H. Intraday shelf replenishment decision support for perishable goods. *Int. J. Prod. Econ.* **231**, 107828 (2021).
- Therkelsen, P., Masanet, E. & Worrell, E. Energy efficiency opportunities in the U.S. commercial baking industry. *J. Food Eng.* **130**, 14–22 (2014).
- Bouman, C. A. *et al.* CLUSTER: An unsupervised algorithm for modeling gaussian mixtures. 20 (1997).
- Babor, M. & Hitzmann, B. Application of nature-inspired multi-objective optimization algorithms to improve the bakery production efficiency. In *ECP 2022* 31 (MDPI, 2022). doi:<https://doi.org/10.3390/ECP2022-12630>.
- Ye, H., Li, W. & Nault, B. R. Trade-off balancing between maximum and total completion times for no-wait flow shop production. *Int. J. Prod. Res.* **58**, 3235–3251 (2020).
- Babor, M. & Hitzmann, B. Small and medium-sized bakery production data for scheduling. (2022) <https://doi.org/10.17632/DHGBSSB8NS.2>.
- Van Rossum, G. & Drake Jr, F. *Python Tutorial; Technical Report CS-R9526*. (Centrum voor Wiskunde en Informatica (CWI): Amsterdam, The Netherlands, 1995).
- Deb, K., Pratap, A., Agarwal, S. & Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* **6**, 182–197 (2002).
- Fonseca, C. M. & Fleming, P. J. Genetic algorithms for multiobjective optimization: Formulation, discussion and generalization (1993).
- Horn, J. D., Nafpliotis, N. & Goldberg, D. A niched Pareto genetic algorithm for multiobjective optimization. In *Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence* (1994). <https://doi.org/10.1109/ICEC.1994.350037>.
- Srinivas, N. & Deb, K. Multiobjective function optimization using nondominated sorting genetic algorithms. *Evol. Comput.* **2**, 1301–1308 (1994).
- Zitzler, E., Deb, K. & Thiele, L. Comparison of multiobjective evolutionary algorithms: Empirical results. *Evol. Comput.* **8**, 173–195 (2000).
- Zitzler, E., Laumanns, M. & Thiele, L. SPEA2: Improving the strength pareto evolutionary algorithm. In: *Computer Engineering and Networks Laboratory (TIK) Department of Electrical Engineering Swiss Federal Institute of Technology (ETH) Zurich ETH Zentrum TIK-Report* 103, (2001).
- Zitzler, E. & Thiele, L. An evolutionary algorithm for multiobjective optimization: The strength pareto approach. *Computer Engineering and Communication Networks Lab TIK, Swiss Federal Institute of Technology ETH* 43 (1998).
- Elhossini, A., Areibi, S. & Dony, R. Strength pareto particle swarm optimization and hybrid EA-PSO for multi-objective optimization. *Evol. Comput.* **18**, 127–156 (2010).
- Emmerich, M. T. M. & Deutz, A. H. A tutorial on multiobjective optimization: Fundamentals and evolutionary methods. *Nat. Comput.* **17**, 585–609 (2018).
- Kukkonen, S. & Lampinen, J. GDE3: The third evolution step of generalized differential evolution. In: *2005 IEEE Congress on Evolutionary Computation*, vol. 1, 443–450 (2005).
- Eberhart, R. & Kennedy, J. A new optimizer using particle swarm theory. In *MHS'95. Proceedings of the Sixth International Symposium on Micro Machine and Human Science* 39–43 (1995). <https://doi.org/10.1109/MHS.1995.494215>.
- Coello, C. A. C., Pulido, G. T. & Lechuga, M. S. Handling multiple objectives with particle swarm optimization. *IEEE Trans. Evol. Comput.* **8**, 256–279 (2004).
- Coello, C. C. & Lechuga, M. S. MOPSO: A proposal for multiple objective particle swarm optimization. In *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No. 02TH8600)*, Vol. 2, 1051–1056 (IEEE, 2002).
- Durillo, J. J. *et al.* Multi-objective particle swarm optimizers: An experimental comparison. In *Evolutionary Multi-criterion Optimization* (eds Ehrgott, M. *et al.*) 495–509 (Springer, 2009). https://doi.org/10.1007/978-3-642-01020-0_39.

41. Hu, X. & Eberhart, R. Multiobjective optimization using dynamic neighborhood particle swarm optimization. In: *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No.02TH8600)*, Vol. 2, 1677–1681 (2002).
42. Nebro, A. J. et al. SMPSO: A new PSO-based metaheuristic for multi-objective optimization. In: *2009 IEEE Symposium on Computational Intelligence in Multi-Criteria Decision-Making (MCDM)* 66–73 (2009). doi:<https://doi.org/10.1109/MCDM.2009.4938830>.
43. Ray, T. & Liew, K. M. A swarm metaphor for multiobjective design optimization. *Eng. Optim.* **34**, 141–153 (2002).
44. Sierra, M. R. & Coello Coello, C. A. Improving PSO-based multi-objective optimization using crowding, mutation and ϵ -dominance. In *Evolutionary Multi-Criterion Optimization* (eds Coello Coello, C. A. et al.) 505–519 (Springer, 2005). https://doi.org/10.1007/978-3-540-31880-4_35.
45. Sun, Y. & Gao, Y. A multi-objective particle swarm optimization algorithm based on Gaussian mutation and an improved learning strategy. *Mathematics* **7**, 148 (2019).
46. Ahmadi, E., Zandieh, M., Farrokh, M. & Emami, S. M. A multi objective optimization approach for flexible job shop scheduling problem under random machine breakdown by evolutionary algorithms. *Comput. Oper. Res.* **73**, 56–66 (2016).
47. Deliktaş, D., Özcan, E., Ustun, O. & Torkul, O. Evolutionary algorithms for multi-objective flexible job shop cell scheduling. *Appl. Soft Comput.* **113**, 107890 (2021).
48. He, L., Cao, Y., Li, W., Cao, J. & Zhong, L. Optimization of energy-efficient open shop scheduling with an adaptive multi-objective differential evolution algorithm. *Appl. Soft Comput.* **118**, 108459 (2022).
49. Li, X. & Ma, S. Multi-objective memetic search algorithm for multi-objective permutation flow shop scheduling problem. *IEEE Access* **4**, 2154–2165 (2016).
50. Santos, V. L. A., Carvalho, T. F. M., de Assis, L. P., Weiss-Cohen, M. & Guimarães, F. G. Multi-objective iterated local search based on decomposition for job scheduling problems with machine deterioration effect. *Eng. Appl. Artif. Intell.* **112**, 104826 (2022).
51. Abido, M. A. & Elazouni, A. Modified multi-objective evolutionary programming algorithm for solving project scheduling problems. *Expert Syst. Appl.* **183**, 115338 (2021).
52. Ye, H., Li, W., Abedini, A. & Nault, B. An effective and efficient heuristic for no-wait flow shop production to minimize total completion time. *Comput. Ind. Eng.* **108**, 57–69 (2017).
53. Ye, H., Li, W. & Abedini, A. An improved heuristic for no-wait flow shop to minimize makespan. *J. Manuf. Syst.* **44**, 273–279 (2017).
54. Jolliffe, I. T. *Principal Component Analysis* (Springer-Verlag, 2002).
55. Babor, M., Pedersen, L., Kidmose, U., Paquet-Durand, O. & Hitzmann, B. Application of non-dominated sorting genetic algorithm (NSGA-II) to increase the efficiency of bakery production: A case study. *Processes* **10**, 1623 (2022).
56. Ye, H., Wang, X. & Liu, K. Adaptive preventive maintenance for flow shop scheduling with resumable processing. *IEEE Trans. Automat. Sci. Eng.* **18**, 106–113 (2021).
57. Miyata, H. H., Nagano, M. S. & Gupta, J. N. D. Integrating preventive maintenance activities to the no-wait flow shop scheduling problem with dependent-sequence setup times and makespan minimization. *Comput. Ind. Eng.* **135**, 79–104 (2019).
58. Storn, R. & Price, K. Differential evolution – A simple and efficient heuristic for global optimization over continuous spaces. *J. Global Optim.* **11**, 341–359 (1997).
59. Kukkonen, S. & Lampinen, J. An Extension of Generalized Differential Evolution for Multi-objective Optimization with Constraints. in *Parallel Problem Solving from Nature - PPSN VIII* (eds Yao, X. et al.) 752–761 (Springer, 2004). doi:https://doi.org/10.1007/978-3-540-30217-9_76.
60. Lampinen, J. A constraint handling approach for the differential evolution algorithm. in *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No.02TH8600)* vol. 2 1468–1473 (2002).

Author contributions

M.B. performed conceptualization, methodology, software, formal analysis, investigation, visualization, writing—original draft preparation. O.P. contributed to conceptualization and methodology. R.K. contributed to methodology and supervised the study. B.H. supervised the study and played the role of project administration. All authors have read and agreed to the published version of the manuscript.

Funding

Open Access funding enabled and organized by Projekt DEAL. This study was funded by EIT Food of the European Institute of Innovation and Technology (EIT), a body of the European Union, the E.U. Framework Program for Research and Innovation for the project entitled “Optimization of bakery processes by a computational tool together with consumer feedback to minimize ecological footprint and food waste: Making baking more efficient”.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to M.B.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2023